

EDB Lasso Diagnostic Reduction Process

Operational Standard Operating Procedure (SOP)

WHY, WHAT, and HOW of Secure Diagnostic Sanitization

WHY

WHAT

HOW

CODE EVOLUTION

 [Download Presentation PDF](#)

Author: Enterprise DBA & Platform Operations

Tool: `lasso_redact.py` Engine (PoC)

The Enterprise Dilemma

When high-severity incidents strike an **EDB Postgres** cluster (failovers, replication lag, memory exhaustion), vendor support asks for an **EDB Lasso bundle**.

Infosec Mandate

- Plaintext passwords in `postgresql.conf`
- MD5 / SCRAM-SHA-256 hashes in `pg_hba.conf`
- Private IP topologies expose internal subnets
- Sharing breaches SOC 2, HIPAA, PCI-DSS

Support Mandate

- Need cluster node relationship topology
- Need Barman & Repmgr configurations
- Need exact error logs & OS metrics
- Cannot debug without realistic context

“ **Resolution:** An automated, verifiable, deterministic redaction pipeline before external transfer. ”

WHY Why Is This Procedure Mandatory?

Every Lasso report captures complete system configuration states:

1. Authentication Secrets:

```
primary_conninfo = 'host=10.0.12.45 user=repuser password=SuperSecret!'
```

2. Database URI Schemes:

```
postgres://admin:P@ssword123@192.168.1.50:5432/mydb
```

3. Portal & API Tokens:

```
EDB_CUSTOMER_TOKEN = edb_prod_cust_tok_...
```

4. Internal Network Maps:

RFC 1918 addresses (`10.x.x.x` , `172.16.x.x` , `192.168.x.x`) that reveal internal VPC architectures.

The Risk: Sending raw bundles exposes live database credentials and corporate network topology to external ticket portals and ticketing archives.

WHAT What Is the Lasso Reduction Process?

A modular 4-stage pipeline that sanitizes bundles while retaining 100% diagnostic integrity:

[Raw Lasso Archive (.tar.gz / .tar.bz2)]



Stage 01: Ingestion & Archive Unpacking

Stage 02: Pattern Masking & Credential Redaction

Stage 03: Deterministic IP Pseudonymization

Stage 04: Audit Manifest Generation & Re-packing



[Sanitized Bundle + lasso_reduction_manifest.json]

`CODE` The Real-World Bug Uncovered

When running early versions against production-like Lasso reports:

```
$ python3 lasso_redact.py -i lassoreport.tar.gz -o lassoreport_redacted.tar.gz
...
[+] Reduction Completed: Total Redacted Secrets: 0
```

Why did it report 0 redactions?

1. Nested Archive Architecture:

EDB Lasso creates an outer `.tar.gz`, but inside it packages individual node diagnostics as **nested `.tar.bz2` archives** (`node-01.tar.bz2`, `node-02.tar.bz2`).

The script was simply copying the inner `.tar.bz2` files without looking inside!

2. Binary File Drops:

When decoding non-UTF8 files (binaries, core dumps, SSL certs), `open()` with `errors="replace"` or unhandled read exceptions skipped or mangled raw binary assets.

CODE Handling Nested .tar.bz2 Archives

The engine now recursively detects and unpacks inner bzip2 archives:

```
def redact_nested_tar_bz2(self, archive_path: Path, output_path: Path):
    """Extract, redact and rebuild nested .tar.bz2 archives."""
    with tempfile.TemporaryDirectory() as tmp_in, tempfile.TemporaryDirectory() as tmp_out:
        with tarfile.open(archive_path, "r:bz2") as tar:
            tar.extractall(tmp_in)

        # Recursively redact the extracted directory tree
        self.redact_directory(Path(tmp_in), Path(tmp_out))

        output_path.parent.mkdir(parents=True, exist_ok=True)
        with tarfile.open(output_path, "w:bz2") as tar:
            tar.add(tmp_out, arcname="")
```

Inside `redact_directory()`:

```
if file_name.endswith(".tar.bz2"):
    print(f"[*] Processing nested archive: {src_file}")
    self.redact_nested_tar_bz2(src_file, dest_file)
    total_files += 1
    continue
```

CODE Preserving Binaries via `shutil.copy2`

In `redact_file()`, text processing is encapsulated in a safe `try-except` block:

```
def redact_file(self, file_path: Path, output_path: Path) -> int:
    """Redacts a single file and writes to destination."""
    try:
        with open(file_path, "r", encoding="utf-8", errors="replace") as f:
            content = f.read()

            redacted_content, matches = self.redact_text(
                content, source_identifier=file_path.name
            )
            output_path.parent.mkdir(parents=True, exist_ok=True)
            with open(output_path, "w", encoding="utf-8") as f:
                f.write(redacted_content)
            return len(matches)

    except Exception:
        # Fallback: preserve binary dumps, certificates, or unparsed files byte-for-byte
        output_path.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(file_path, output_path)
        return 0
```

<header>Operational Procedure | The HOW (Step-by-Step SOP)</header>

HOW Standard Operating Procedure (SOP)

Phase 1: Bundle Capture (DBA Host)

```
# Generate the official EDB Lasso bundle
sudo lasso --barman --system --postgresql --tarball
# Creates: /tmp/edb-lasso-report-<cluster>-<date>.tar.gz
```

Phase 2: Reduction Execution (Secure Staging)

```
python3 lasso_redact.py \
-i /tmp/edb-lasso-report.tar.gz \
-o /tmp/edb-lasso-report-sanitized.tar.gz
```

You will see:

```
[*] Processing nested archive: ../node1.tar.bz2
```

```
[+] Reduction Completed Successfully! Total Redacted Secrets: 21
```

HOW Validating the Sanitized Output

Phase 3: Inspect Audit Manifest

```
cat sanitized_bundle/lasso_reduction_manifest.json | jq .category_breakdown
```

Sample audit output:

```
{
  "title": "EDB Lasso Reduction Manifest",
  "total_redactions": 21,
  "category_breakdown": {
    "network": 13,
    "credentials": 6,
    "api_tokens": 2
  },
  "pseudonymized_ip_count": 6
}
```

“**Infosec Rule:** If `"total_redactions": 0` on an archive containing secrets, abort transmission and investigate new pattern rules.

Deterministic IP Pseudonymization

Why not simply replace all IPs with `XXX.XXX.XXX.XXX` ?

- PostgreSQL replication topologies require knowing which replica talks to which primary.
- Random hashing breaks replication correlation across configuration files.

Deterministic Solution:

```
10.0.12.45 (Primary)  → PSEUDO_IP_NODE_01 (Consistently throughout all logs)
10.0.12.46 (Replica) → PSEUDO_IP_NODE_02 (Consistently throughout all logs)
192.168.1.50 (Barman) → PSEUDO_IP_NODE_03 (Consistently throughout all logs)
```

Result: Support can reconstruct the failover timeline and network flow without ever learning the customer's actual private IP space.

Pre-Transmission Checklist for DBAs

Phase	Step	Command / Check
1. Verification	Run Unit Test Suite	<code>python3 test_lasso_redact.py</code> (Must pass 8/8)
2. Reduction	Run Engine on Tarball	<code>python3 lasso_redact.py -i <in> -o <out></code>
3. Nested Check	Observe Node Logs	Ensure <code>[*] Processing nested archive:</code> appeared
4. Manifest	Inspect Counts	Confirm <code>total_redactions > 0</code>
5. Evidence	Attach Manifest	Upload <code>lasso_reduction_manifest.json</code> with ticket
6. Cleanup	Secure Staging Cleanup	Remove raw bundles from temporary directory

Summary & Key Takeaways

1. **Compliance by Design:** Eliminates password leaks and network exposure before files leave your perimeter.
2. **Recursive Archive Support:** Handles both outer `.tar.gz` and nested `.tar.bz2` node bundles.
3. **Data Integrity Guarantee:** `shutil.copy2` fallback protects binary dumps and certificates from corruption.
4. **Transparent Audit Trail:** Cryptographic and categorical manifest satisfies SOC 2 / ISO 27001 evidence requirements.

Links & References

- **Download Slide Deck (PDF):** Download `presentation.pdf`
- **Web Dashboard:** <http://localhost:30088/index.html>
- **Interactive Presentation:** <http://localhost:30088/presentation.html>
- **Source Code:** [`lasso_redact.py`](file:///Users/rifaterdemsahin/projects/edb-lasso-reduction-process/lasso_redact.py)
- **Test Suite:** [`test_lasso_redact.py`](file:///Users/rifaterdemsahin/projects/edb-lasso-reduction-process/test_lasso_redact.py)
- **GitHub Repository:** <https://github.com/rifaterdemsahin/edb-lasso-reduction-process>